

Cursorを使ったAIコーディング体験会

～AIとのペアプログラミングを体験しよう～

2025年06月19日（木）
株式会社ネクストビート

本日の流れ (18:30 - 20:00)

時間	内容
18:30-18:40	会社紹介
18:40-18:50	講義：生成AI・AIエージェント・Cursor入門
18:50-19:00	Cursor 環境設定・基本操作
19:00-19:20	演習：簡易タスクリストアプリ - HTML/JS作成
19:20-19:30	休憩
19:30-19:50	演習：簡易タスクリストアプリ - 機能追加/スタイリング
19:50-20:00	まとめ・質疑応答

第1部：講義 (18:40 - 18:50)

生成AI・AIエージェント・Cursor入門

1. AIコーディングの波

- LLM (大規模言語モデル) の進化により、AIがコードを生成・理解・編集する時代へ。
- ソフトウェア開発の生産性向上、新しい働き方を実現する可能性。
- **Cursor** のようなツールを使いこなし、AIを開発のパートナーに！

2. 生成AIとAIエージェント

- **生成AI (Generative AI):** 新しいコンテンツ (テキスト, 画像, コード) を生成するAI。
- **AIエージェント (AI Agent):** 自律的にタスクを実行するAI。コーディング支援も高度化。
 - 例: Cursor, GitHub Copilot (支援型) ～ Claude Code, Codex, Jules, Devin (自律型)

3. Cursorとは？ ✨ AIファーストなコードエディタ

- VS Codeベースで馴染みやすいインターフェース。
- **AIとの対話 (`Ctrl+L` / `Cmd+L`)** や **インライン指示 (`Ctrl+K` / `Cmd+K`)** でコーディングを加速。
- コード生成、リファクタリング、デバッグ、ドキュメント作成などをAIがサポート。
- **無料版**でも強力な機能を体験可能！

第2部：Cursor 環境設定・基本操作 (18:50 - 19:00)

Cursorの準備はOK？

- お手元のPCに **Cursor** はインストール済みですか？
 - 最初の起動まで済ませておいてください。
- Wi-Fi接続を確認してください (ゲストWi-Fiあり)。
- 困ったら遠慮なくスタッフに声をかけてください！

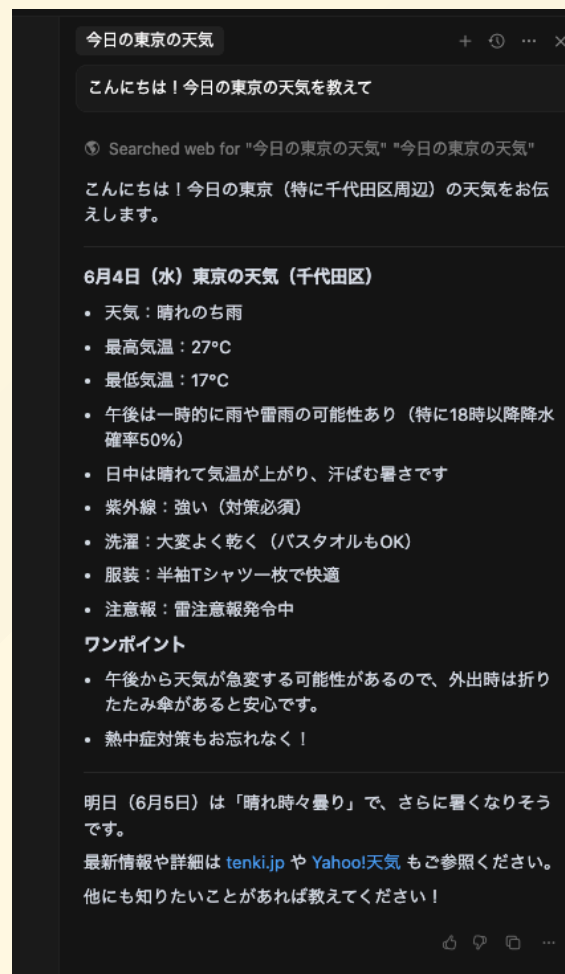
基本的な使い方 (おさらい)

- AIチャットを開く/閉じる:
 - Windows: `Ctrl + L`
 - Mac: `Cmd + L`
- 選択範囲やカーソル位置でAIに指示 (インライン編集/生成):
 - Windows: `Ctrl + K`
 - Mac: `Cmd + K`

試しにチャットでAIに「`こんにちは！今日の東京の天気を教えて`」と話しかけてみましょう。

Cursorに質問した結果

- 回答は毎回変わります

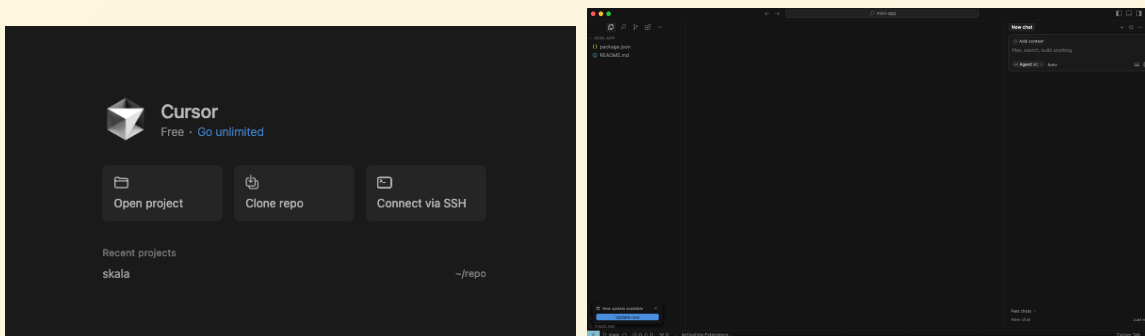


リポジトリをクローンする

**GitHubリポジトリ `mini-app` をクローンして、演習用のコードベースを準備します

```
git clone https://github.com/nextbeat-dev/mini-app.git
```

1. 「Open project」 をクリック
2. クローンした `mini-app` フォルダを選択



第3部：演習 (19:00 - 19:20)

簡易タスクリストアプリ

演習：目標

HTML、CSS、JavaScriptを使ったシンプルなタスクリストアプリをCursorと一緒に作ってみましょう！

- **機能要件：**
 1. タスクを入力できるフォームがある。
 2. 入力されたタスクがリスト表示される。
 3. タスクを完了状態にできる (打ち消し線など)。
 4. タスクを削除できる。

演習：プロンプト例（HTML作成）

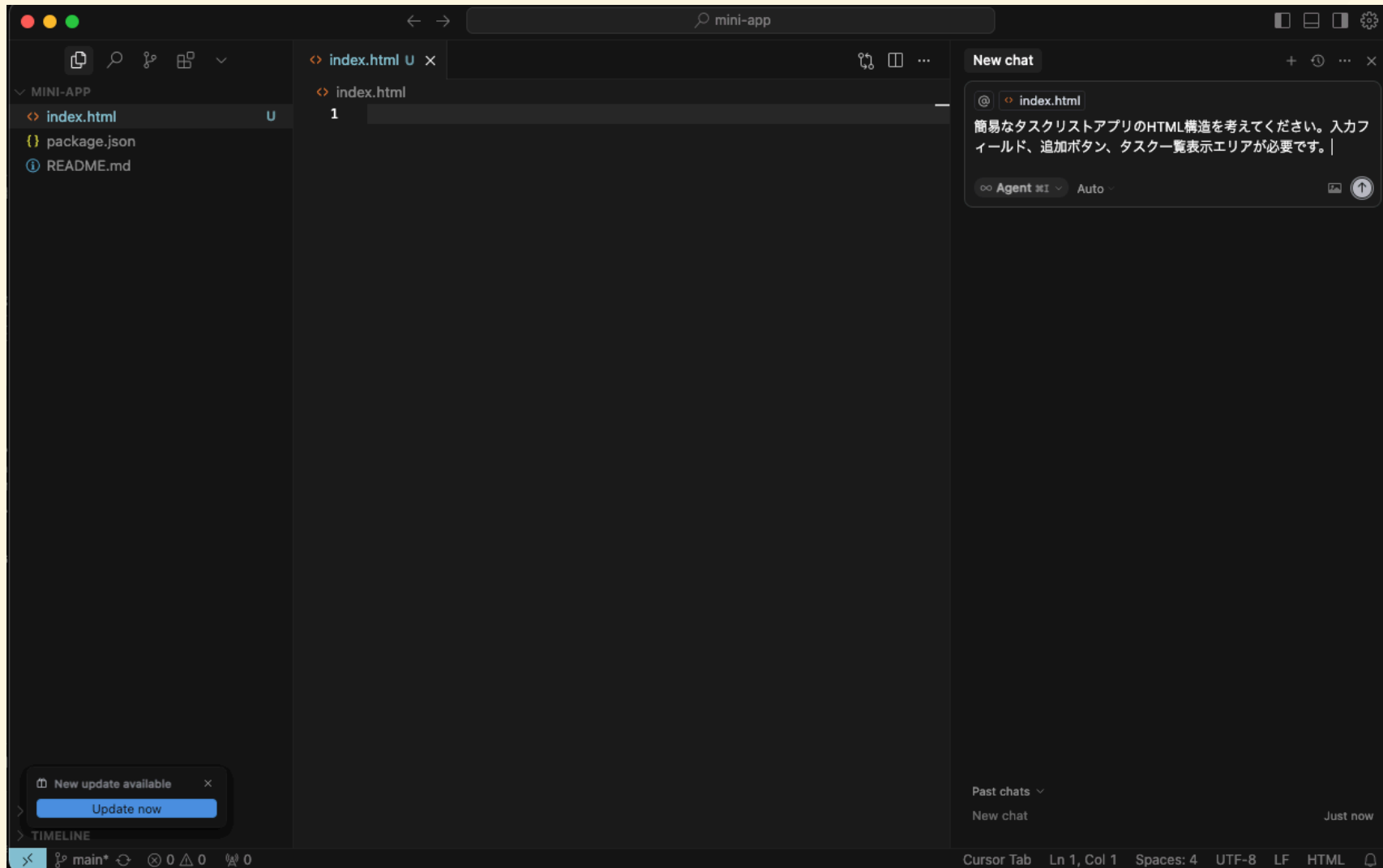
HTMLの骨組み作成

- `index.html` ファイルを作成。
- Cursorにチャットで依頼:

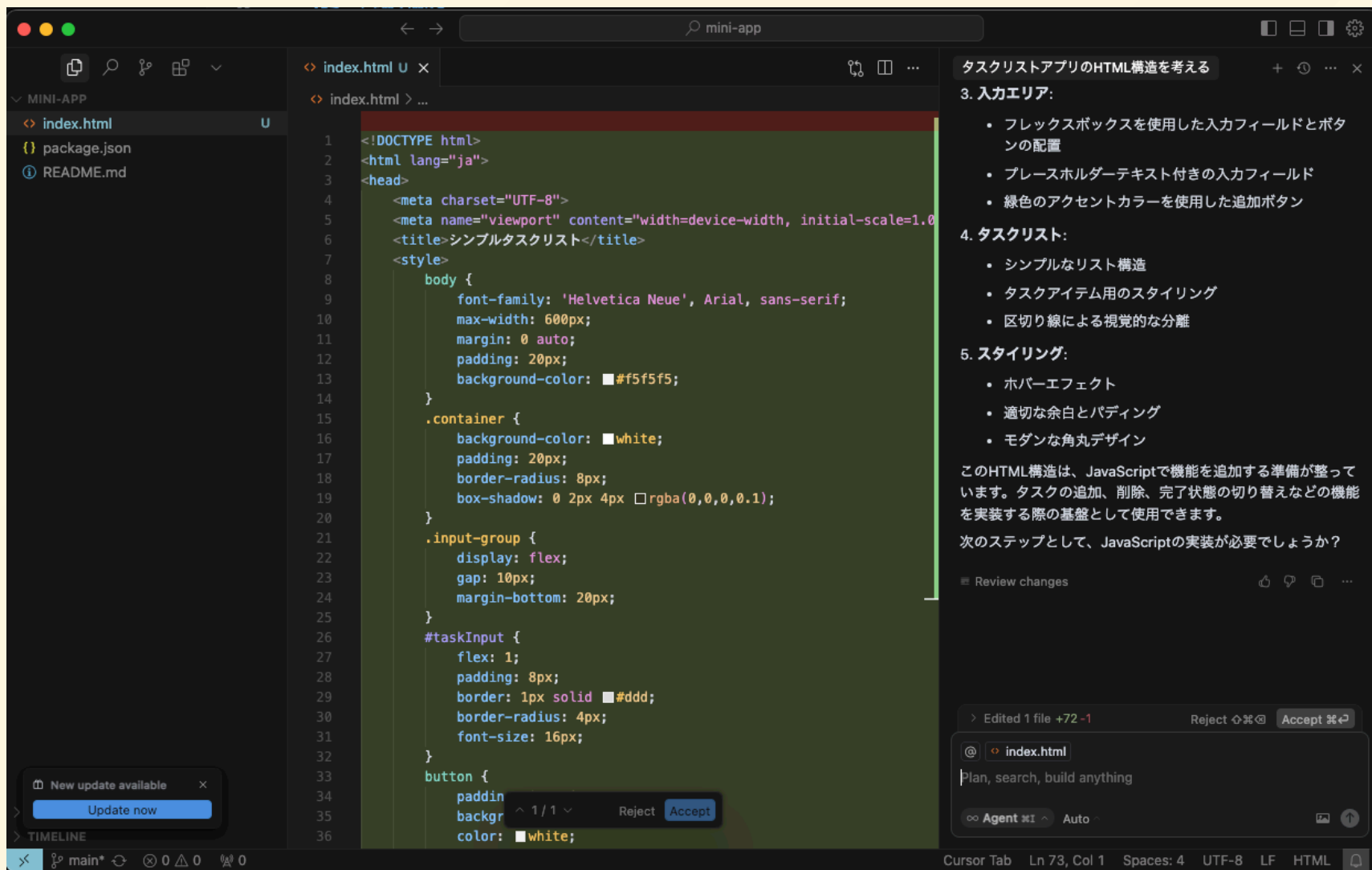
簡単なタスクリストアプリのHTML構造を考えてください。
入力フィールド、追加ボタン、タスク一覧表示エリアが必要です。

- Cursorが生成したHTMLを確認し、修正が不要なら「accept」をクリック。

演習：スクリーンショット（HTML作成プロンプト）



演習：スクリーンショット（HTML作成結果）



演習：プロンプト例（.jsファイル作成）

基本的なJavaScriptのロジック (タスク追加と表示)を作成

- `script.js` ファイルを作成し、HTMLから読み込む（JSファイルは空）。

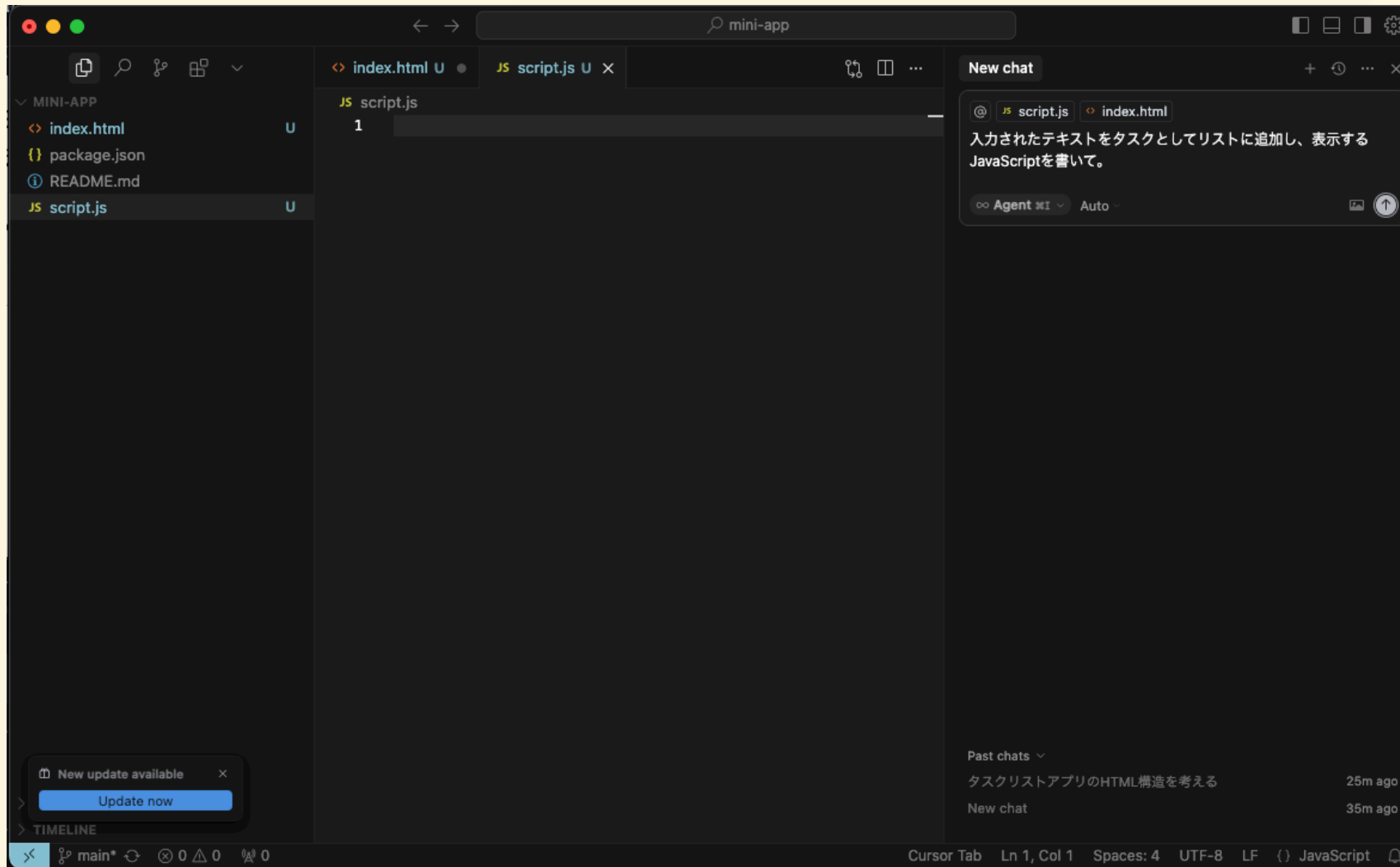
```
<head>
  ...
  <script src="script.js" defer></script>
</head>
```

- Add context -> Files and Foldersから追加:
 - `index.html`, `script.js` を選択
- Cursorにチャットやインライン編集で依頼:

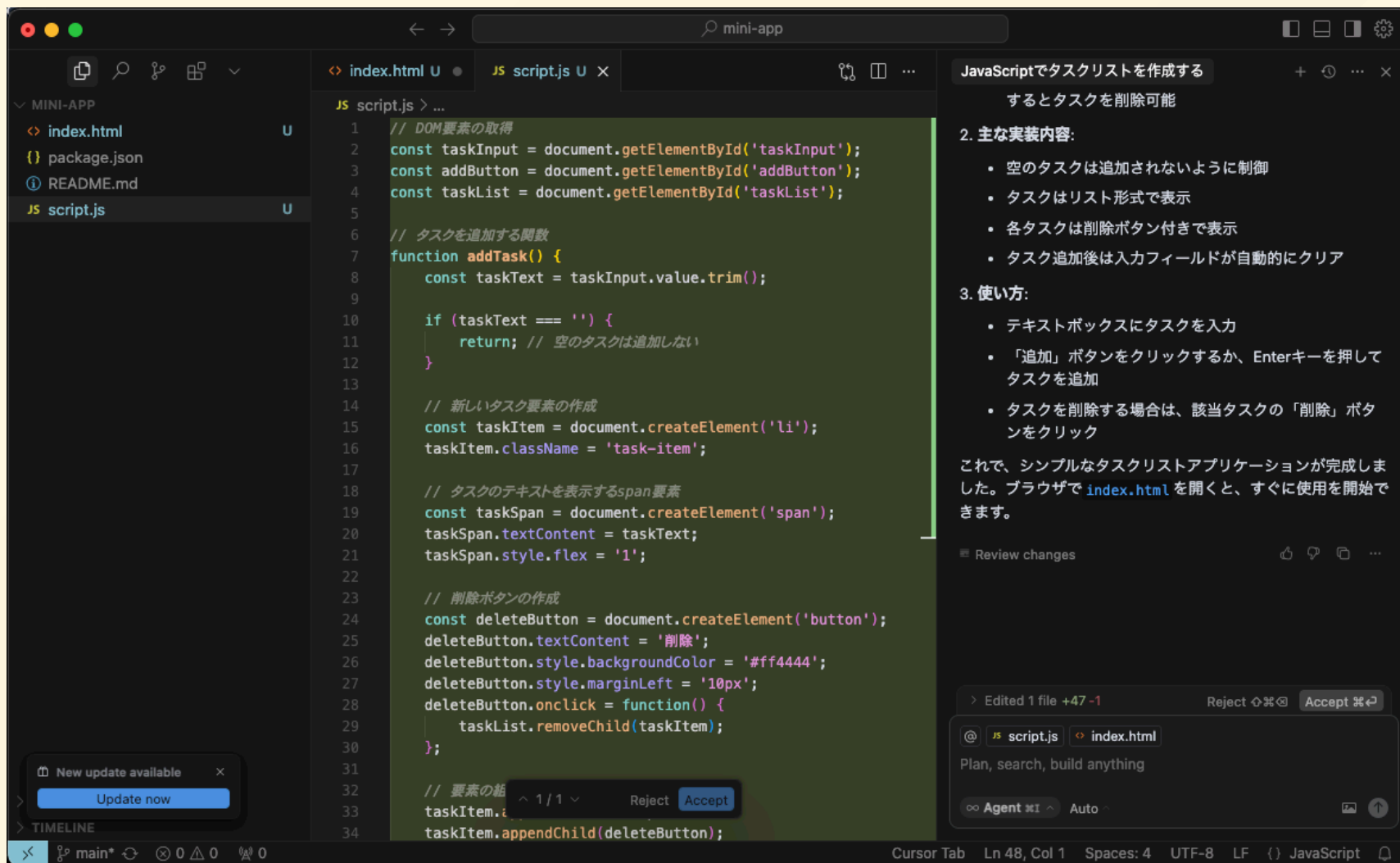
入力されたテキストをタスクとしてリストに追加し、表示するJavaScriptを書いて。

- Cursorが生成したJavaScriptを確認し、修正が不要なら「accept」をクリック。

演習：スクリーンショット（JavaScript作成プロンプト）



演習：スクリーンショット（JavaScript作成結果）



演習：HTMLを開く

- `index.html` ファイルをChromeなどのブラウザで開いて、タスクリストアプリの基本的な動作を確認。

演習：スクリーンショット（タスクリストアプリの動作確認）

タスクリスト

新しいタスクを入力...

追加

疲れたので寝る

削除

休憩 (19:20 - 19:30)

 ごゆっくりどうぞ！

第4部：演習 (19:30 - 19:50)

簡易タスクリストアプリ - 機能追加/スタイリング

演習：プロンプト例 (タスクの完了・削除機能)

- Cursorに機能追加を依頼:

タスクに完了チェックボックスを付けて、チェックされたら打ち消し線を入れるようにして。
各タスクに削除ボタンも付けて、クリックしたらそのタスクを削除できるようにして。

- Cursorが生成したコードを確認し、必要に応じて修正。

演習：スクリーンショット（修正プロンプト）

The screenshot shows the Cursor IDE interface. The main editor displays the `index.html` file with the following content:

```
<html lang="ja">
<head>
  <style>
    button {
    }
    button:hover {
      background-color: #45a049;
    }
    #taskList {
      list-style: none;
      padding: 0;
    }
    .task-item {
      display: flex;
      align-items: center;
      padding: 10px;
      border-bottom: 1px solid #eee;
    }
    .task-item:last-child {
      border-bottom: none;
    }
  </style>
  <script src="script.js" defer></script>
</head>
<body>
  <div class="container">
    <h1>タスクリスト</h1>
    <div class="input-group">
      <input type="text" id="taskInput" placeholder="新しいタスクを入力">
      <button id="addButton">追加</button>
    </div>
    <ul id="taskList">
      <!-- タスクはJavaScriptで動的に追加されます -->
    </ul>
  </div>
</body>
</html>
```

The sidebar on the left shows the file explorer with the following files:

- index.html
- package.json
- README.md
- script.js

The right-hand pane displays a prompt and explanation for the code changes:

JavaScriptでタスクリストを作成する

- これにより、HTMLの要素が完全に読み込まれた後にJavaScriptが実行されるようになります

2. なぜこの修正が必要だったのか:

- 以前のコードでは、HTMLの要素が読み込まれる前にJavaScriptが実行されていました
- そのため、`getElementById` で要素が見つからず、`null` が返されていました
- `null` に対して `addEventListener` を呼び出そうとしたため、エラーが発生していました

3. 修正後の動作:

- ページの読み込みが完了してからJavaScriptが実行されるため、エラーは発生しなくなります
- タスクの追加や削除が正常に機能ようになります

これで、エラーは解消され、タスクリストアプリケーションが正常に動作するはずです。

Review changes

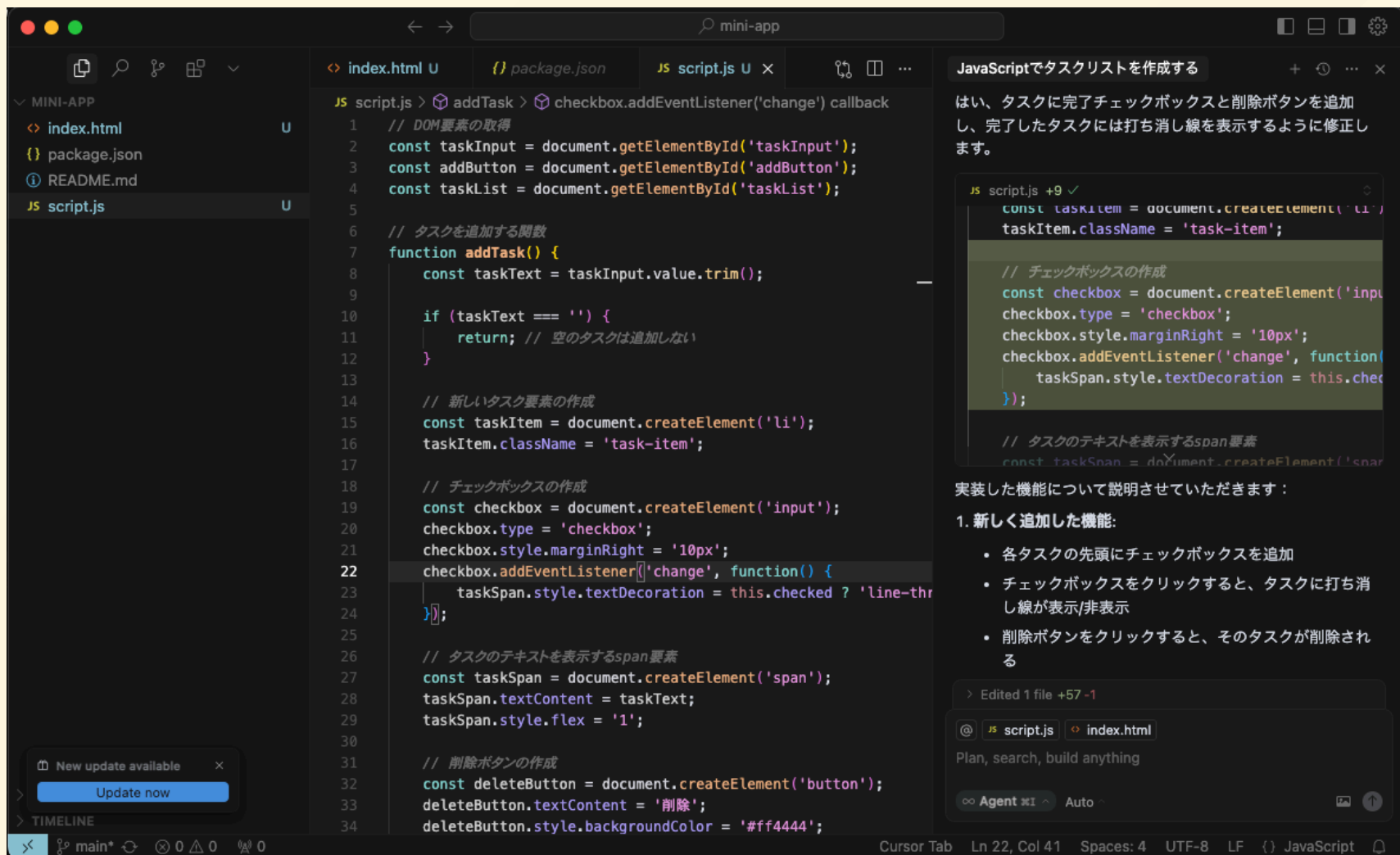
Edited 1 file +51 -1

@ JS script.js index.html

タスクに完了チェックボックスを付けて、チェックされたら打ち消し線を入れるようにして。
各タスクに削除ボタンも付けて、クリックしたらそのタスクを削除できるようにして。

Agent AI Auto

演習：スクリーンショット（修正結果）



演習：タスクリストアプリの動作確認

- 再度 `index.html` をブラウザで開いて、タスクの完了・削除機能が動作することを確認

タスクリスト

追加

☒ 疲れたので寝る

削除

演習：プロンプト例 (CSSでスタイリング)

- `style.css` ファイルを作成し、HTMLから読み込む
 - `<link rel="stylesheet" href="style.css">` を `<head>` 内に追加
- Add context -> Files and Foldersから追加:
 - `style.css` を選択
- Cursorにデザインを依頼:

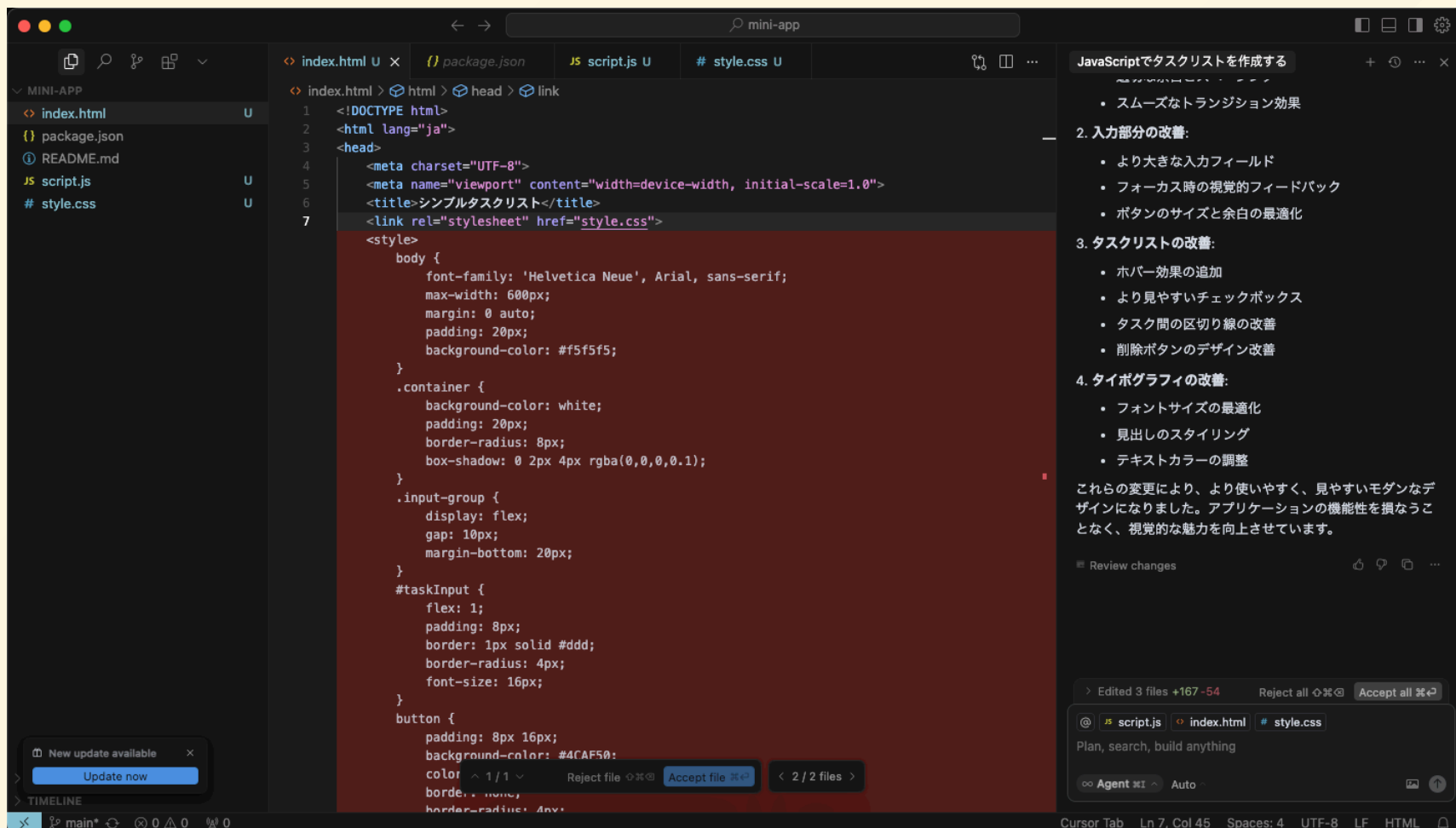
このタスクリストアプリをもう少し見やすくスタイリングしてください。
モダンでシンプルな感じで。

ヒント： 生成されたコードが期待通りでなければ、AIに修正を依頼したり、自分で少し手直ししてみましょう！

演習：スクリーンショット（CSS作成プロンプト）

 CSS生成プロンプト

演習：スクリーンショット（CSS作成結果）



演習：HTMLを開く

- `index.html` ファイルを再度ブラウザで開いて、タスクリストアプリの見た目や動作を確認。

演習：スクリーンショット（タスクリストアプリの動作確認）



- 配色などが微妙に変わっているのがわかりますね！

本日のまとめ

- AIコーディングツール **Cursor** の基本的な使い方を体験しました。
- AIとの対話を通じて、簡単なWebアプリケーションを作成しました。
 - プロンプトによるコード生成
 - 既存コードの編集、機能追加
- AIは開発の強力な **アシスタント** になり得ます。
 - 定型作業の自動化、アイデアの壁打ち、新しい技術の学習など。

今後の開発にもAIツールを積極的に活用してみてください！

質疑応答

イベントに関するご質問、CursorやAIコーディングに関するご質問など、どうぞ！

懇親会のご案内

- **20:00から9Fにて懇親会**を開催します！（任意参加）。

本日はご参加いただき、ありがとうございました！

アンケートにご協力ください（もしあればQRコードなど）